

SYSTEM DESIGN

10 Mock Interview Practice Problems

Book 4 of the System Design Roadmap

Full worked answers using the 5-step interview framework

Best used after Books 1-3 (Days 1-30). Try solving each problem yourself first, then compare against the model answer.

The 10 Problems

- | | |
|----|--|
| 1 | Design a URL Shortener (bit.ly) |
| 2 | Design a Rate Limiter |
| 3 | Design a Twitter/X-Style News Feed |
| 4 | Design Instagram (Photo Sharing at Scale) |
| 5 | Design WhatsApp (Real-Time Chat Messaging) |
| 6 | Design Uber (Ride-Matching at Scale) |
| 7 | Design YouTube (Video Upload & Streaming) |
| 8 | Design a Web Crawler |
| 9 | Design Dropbox (File Storage & Sync) |
| 10 | Design a Ticket Booking System (BookMyShow/Ticketmaster) |

How to use this book: for each problem, cover the model answer, set a 30-40 minute timer, and try solving it yourself out loud (or on paper/whiteboard) using the 5-step framework from Day 10. Only then read the model answer and compare your approach. The goal isn't to memorize these exact answers - interviewers can tell - it's to internalize the framework so you can apply it to any new problem you're given.

The 5-Step Framework (Recap)

- **1. Clarify requirements** - scope, scale, and what matters most.
- **2. Estimate capacity** - rough numbers for traffic and storage.
- **3. High-level design** - the major building blocks and data flow.
- **4. Deep dive** - the one or two hardest parts, in detail.
- **5. Bottlenecks & trade-offs** - what could break, and how you'd fix it.

PROBLEM 1

Design a URL Shortener (bit.ly)

A classic warm-up problem that tests the fundamentals cleanly

■ Clarifying Questions to Ask First

How many links are created per day? Do links expire? Can users pick a custom short code? Do we need click analytics? For this answer, assume 100 million new links/day, no custom codes required, and basic click-count analytics.

Requirements & Capacity

Core requirement: given a long URL, return a short one; given a short URL, redirect to the original. At 100 million new links/day, that's roughly 1,200 writes/second on average. Since redirects happen far more often than link creation, reads could easily be 10-100x that - so this is a heavily read-dominated system.

High-Level Design

Client submits a long URL to a load balancer, which routes it to an application server. The server generates a unique short code, stores the mapping (short_code -> long_url) in a database, and returns the short link. On visiting a short link, the server looks up the mapping and issues an HTTP redirect.

Deep Dive: Generating Unique Short Codes

Use a distributed counter (or pre-allocated ranges of IDs per server) to generate a unique numeric ID for each new link, then encode that number in base62 (a-z, A-Z, 0-9) to produce a short, compact string like "aZ3dQ1". This avoids collisions without needing to check the database for uniqueness on every single write.

■ Key Trade-off

A random-hash approach (hashing the long URL) is simpler but risks collisions requiring retries. A counter-based approach guarantees uniqueness but requires coordinating ID ranges across multiple application servers so two servers never hand out the same ID.

Bottlenecks & Fixes

- Redirect reads dominate traffic -> add an in-memory **cache** in front of the database for popular short codes.
- Single database becomes a bottleneck at scale -> add **read replicas**; shard by short code if storage alone becomes too large for one server.
- Protect against abuse -> add **rate limiting** on link creation per user/IP.

■ What Interviewers Are Really Testing

This problem tests whether you can reason clearly about read-heavy vs write-heavy traffic, and whether you know how to generate unique IDs at scale without a central bottleneck.

PROBLEM 2

Design a Rate Limiter

A focused, algorithm-heavy problem often asked as a component of a larger system

■ Clarifying Questions to Ask First

Is this a rate limiter for one server or a distributed system with many servers? What's the limiting key - per user, per IP, per API key? What should happen when a client is rate-limited - reject immediately, or queue? Assume here: a distributed rate limiter, keyed by API key, that rejects immediately with an error once the limit is hit.

High-Level Design

Place the rate limiter at the API gateway, before requests reach any backend service - this rejects excess traffic as early and cheaply as possible. Since there are many application servers, the rate-limit counters must be shared across all of them, so a fast, centralized store like Redis is used to track request counts per client.

Deep Dive: Choosing an Algorithm

A **token bucket** algorithm works well here: each API key has a bucket (stored in Redis) that holds up to N tokens and refills at a steady rate. Every request atomically checks and decrements a token; if none remain, the request is rejected. Redis's atomic increment/decrement operations make this safe even with many application servers hitting the same counter simultaneously.

■ Key Trade-off

A fixed window counter is simple but allows a burst of 2x the limit right at the boundary between two windows. A sliding window is smoother and fairer but slightly more complex and costly to compute. Token bucket is a good practical middle ground and is what most production systems actually use.

Bottlenecks & Fixes

- The shared Redis store becomes a single point of failure -> run it as a small, replicated cluster.
- Cross-datacenter deployments add network latency to every check -> consider approximate, per-region counters that sync periodically, trading perfect accuracy for speed.

■ What Interviewers Are Really Testing

This problem tests precise algorithmic thinking (which rate-limiting algorithm, and why) plus an understanding of shared state in a distributed system - a very common combination in interviews.

PROBLEM 3

Design a Twitter/X-Style News Feed

Tests fan-out strategies and handling wildly uneven load (celebrity accounts)

■ Clarifying Questions to Ask First

How many users, and how many people does the average user follow? Does the feed need to be real-time, or is a few seconds of delay acceptable? Assume 500 million users, average 200 follows per user, and a feed that can be a few seconds behind real-time.

Deep Dive: Fan-out on Write vs Fan-out on Read

Fan-out on write: when a user posts, immediately push that post into the pre-built feed of every one of their followers. This makes reading a feed instant (it's already built), but if a celebrity with 50 million followers posts, that single post triggers 50 million writes at once.

Fan-out on read: instead of pre-building feeds, build a user's feed on the fly when they open the app, by pulling recent posts from everyone they follow. This avoids the celebrity write-storm problem, but makes every single feed load more expensive to compute.

The practical answer: use a hybrid. Fan-out on write for regular users (most posts), but for accounts with a huge number of followers, skip the fan-out and merge their posts into followers' feeds at read-time instead.

■ Key Trade-off

Fan-out on write optimizes for read speed at the cost of write amplification. Fan-out on read optimizes for write simplicity at the cost of slower reads. The hybrid approach is more complex to build, but avoids the worst case of either pure strategy.

Supporting Building Blocks

- Posts are **sharded** by user ID across many database servers.
- Pre-built feeds are stored in a fast **cache** (Redis) as a list of recent post IDs, refreshed via the fan-out process.
- A **message queue** handles the fan-out work asynchronously so posting never blocks on updating millions of followers' feeds.

■ What Interviewers Are Really Testing

This is one of the best problems for showing you understand that a single strategy rarely works for every user - real systems often need a hybrid approach to handle extreme, uneven load gracefully.

PROBLEM 4

Design Instagram (Photo Sharing at Scale)

Tests media storage, CDN delivery, and feed generation together

■ Clarifying Questions to Ask First

Are we designing just photo upload/viewing, or also stories, DMs, and comments? Assume here: photo upload, a following-based feed, likes, and comments, at a scale of 500 million daily active users.

High-Level Design

When a user uploads a photo, the app server stores the raw image in **object storage**, generates several resized versions (thumbnail, medium, full-size) in the background, and saves metadata (caption, uploader, timestamp) in a database. The image itself is then served to viewers through a **CDN**, not directly from the origin storage, so it loads quickly worldwide.

Deep Dive: The Feed

Similar to Problem 3, feed generation uses a hybrid fan-out strategy. Metadata for feed posts (post ID, uploader, timestamp) is cached, and the app fetches the actual images from the CDN only when the user scrolls to them - this avoids downloading thousands of images upfront, loading only what's actually visible on screen.

Handling Likes at Scale

A viral post can receive millions of likes within minutes. Rather than updating a single "like count" row in the database on every single like (which would create a massive write bottleneck on one row), likes are typically batched - counted approximately in a fast in-memory store and periodically flushed to the database, so the displayed count may lag by a few seconds under extreme load, but the system never buckles.

■ Key Trade-off

Showing an exactly accurate, real-time like count on every post would require serializing writes to a single counter - a severe bottleneck for viral content. Accepting a few seconds of staleness in exchange for near-infinite write scalability is the trade-off virtually every large platform makes.

■ What Interviewers Are Really Testing

This problem tests whether you can combine multiple building blocks (object storage, CDN, caching, feed fan-out) into one coherent design, and whether you recognize where strict accuracy can be safely traded for scale.

PROBLEM 5

Design WhatsApp (Real-Time Chat Messaging)

Tests persistent connections, delivery guarantees, and offline handling

■ Clarifying Questions to Ask First

One-to-one chat only, or also group chats? Do we need delivery/read receipts? Assume one-to-one and group chat, with delivery and read receipts, at a scale of 2 billion users.

High-Level Design

Unlike a typical request-response API, chat needs a persistent, two-way connection so the server can push new messages to a user instantly. Clients connect to **chat servers** via WebSockets. A **load balancer** distributes these persistent connections across many chat servers, and a lookup service tracks which specific chat server each user is currently connected to.

Deep Dive: Handling Offline Users

When User A sends a message to User B, the server checks if B is currently online (connected to some chat server). If online, the message is pushed instantly through B's connection. If offline, the message is placed in a **per-user message queue** and stored durably; when B reconnects, all queued messages are delivered immediately, then the queue is cleared.

Group Chats & Delivery Receipts

A group message is fanned out to every group member's individual queue/connection, similar to the feed fan-out problem, but typically at a much smaller scale (groups rarely exceed a few hundred members). Delivery and read receipts are small separate events sent back to the original sender, updated asynchronously so they never slow down the actual message delivery.

■ Key Trade-off

Guaranteeing messages are never lost, never duplicated, and always delivered in order across millions of concurrent connections is genuinely hard. Most systems accept "at-least-once" delivery (occasionally a message might be processed twice) and use a unique message ID on the client side to de-duplicate, rather than chasing a perfect "exactly-once" guarantee that's very costly to build.

■ What Interviewers Are Really Testing

This problem tests your understanding of persistent connections (not just request-response), message durability for offline users, and realistic delivery guarantees - a different flavor of scaling problem than a typical read-heavy feed.

PROBLEM 6

Design Uber (Ride-Matching at Scale)

Tests geospatial data, real-time location updates, and matching algorithms

■ Clarifying Questions to Ask First

Do we need to design pricing/surge logic too, or just the core matching of rider to nearby driver? Assume here: core location tracking and matching, at a scale of millions of active drivers worldwide, each sending a location update every few seconds.

Deep Dive: Storing & Querying Location Data

The core challenge is answering "which drivers are near this rider?" quickly, across millions of constantly-moving points. Storing raw latitude/longitude and scanning for nearby drivers would be far too slow. Instead, the world map is divided into a grid of cells (using a technique like **geohashing** or a quadtree), and each driver's current cell is stored in a fast in-memory store. Finding nearby drivers becomes a quick lookup of a rider's cell and its immediate neighboring cells, instead of a search across the entire map.

High-Level Flow

- Driver apps continuously send location updates to a **location service**, which updates the driver's current grid cell in a fast in-memory store.
- A rider requests a ride; the **matching service** looks up nearby drivers using the grid, ranks them by distance/ETA, and sends ride requests to the closest few.
- Once a driver accepts, a dedicated trip session begins, streaming live location updates to the rider via a persistent connection (similar to Problem 5's approach).

■ Key Trade-off

A finer grid gives more precise "nearby" matches but means more cells to check near grid boundaries and more storage overhead. A coarser grid is cheaper to maintain but may consider drivers "nearby" when they're actually a significant distance away. Grid size is tuned based on city density.

Bottlenecks & Fixes

- Extremely dense cities (dense grid cells) can create hot spots -> sub-divide dense cells further, or shard the location store by geographic region.
- Driver location updates are extremely frequent -> write them to an in-memory store, not directly to a durable database, and only persist trip data (not every raw ping) for long-term storage.

■ What Interviewers Are Really Testing

This problem tests geospatial data structures specifically - most candidates haven't thought about "nearby" queries at scale before, so demonstrating you know grid/geohash-based approaches stands out.

PROBLEM 7

Design YouTube (Video Upload & Streaming)

Tests large file handling, transcoding pipelines, and adaptive streaming

■ Clarifying Questions to Ask First

Focus on upload + playback, or also recommendations/comments? Assume core upload and adaptive playback at massive scale, with recommendations treated as a separate downstream system.

Deep Dive: The Upload & Transcoding Pipeline

A raw uploaded video is first stored in **object storage**. Because different viewers have different devices and network speeds, the video can't just be served in its original format - it needs to be **transcoded** into multiple resolutions and formats (e.g., 240p, 480p, 1080p, 4K). This transcoding is a slow, CPU-heavy background job, so the upload flow drops a "transcode this video" message onto a **message queue**, and a separate fleet of transcoding workers processes these jobs asynchronously, notifying the system once each resolution is ready.

Adaptive Streaming

Videos are split into small chunks (a few seconds each) at each resolution. As a user watches, the video player continuously monitors the current network speed and requests the next chunk at whichever resolution best fits that speed - seamlessly stepping down to a lower resolution if the connection slows, rather than buffering. Chunks are served through a **CDN** so playback starts quickly no matter where the viewer is located.

■ Key Trade-off

Transcoding every uploaded video into many resolutions immediately is expensive in compute and storage, especially for videos that get very few views. Many platforms transcode only the most common resolutions immediately, generating rarer/higher resolutions on demand or lazily to balance cost against user experience.

Bottlenecks & Fixes

- Transcoding is CPU-heavy and slow -> scale the transcoding worker fleet independently and horizontally, since it's decoupled from the upload path via the queue.
- Popular videos generate massive read traffic -> lean heavily on CDN caching, since video chunks are immutable once created and perfect for caching.

■ What Interviewers Are Really Testing

This problem tests asynchronous pipeline design (upload now, process later) and adaptive delivery - showing you understand that large media files need fundamentally different handling than small API responses.

PROBLEM 8

Design a Web Crawler

Tests graph traversal at scale, politeness policies, and de-duplication

■ Clarifying Questions to Ask First

Are we crawling the whole web, or a bounded set of sites? Do we need to respect robots.txt and avoid hammering any single site? Assume a general-purpose crawler covering billions of pages, that must be "polite" to the sites it visits.

High-Level Design

The crawler maintains a large **URL frontier** (a queue of URLs waiting to be crawled), starting from a set of seed URLs. Worker processes pull a URL from the frontier, download the page, extract its text and any new links found on it, store the content, and push the newly discovered URLs back onto the frontier - repeating this process indefinitely.

Deep Dive: Avoiding Duplicate Work & Being Polite

With billions of pages, the same URL can be discovered many times from different pages linking to it. A **Bloom filter** gives a fast, memory-efficient first check for "have we already queued or crawled this URL?" before doing an expensive full lookup - dramatically cutting down duplicate crawling. To avoid overwhelming any single website, URLs are grouped by domain, and a per-domain rate limit (plus a delay based on robots.txt rules) ensures the crawler never sends too many requests to one site too quickly.

■ Key Trade-off

Crawling breadth-first (widely, across many domains at once) surfaces a broad snapshot of the web quickly but may miss deep content on any one site. Crawling depth-first thoroughly covers individual sites but is slower to build broad coverage. Most large crawlers use a prioritized frontier that balances both, favoring important or frequently-updated pages.

Bottlenecks & Fixes

- The URL frontier itself can become enormous -> shard it across many machines, partitioned by domain, so per-domain politeness rules are easy to enforce locally.
- Malicious or broken pages (infinite redirect loops, spider traps) can waste crawler resources -> set strict limits on redirect depth and crawl time per page.

■ What Interviewers Are Really Testing

This problem tests graph-traversal thinking at scale plus real-world constraints (politeness, duplicate detection) that go beyond textbook graph algorithms - showing practical engineering judgment, not just theory.

PROBLEM 9

Design Dropbox (File Storage & Sync)

Tests chunking, deduplication, and multi-device conflict resolution

■ Clarifying Questions to Ask First

Do we need real-time sync across multiple devices, and version history? Assume yes to both, at a scale of hundreds of millions of users, many syncing large files across multiple devices.

Deep Dive: Chunking & Deduplication

Instead of uploading an entire file every time it changes, files are split into small fixed-size **chunks**. When a file changes, only the chunks that actually changed need to be re-uploaded and re-synced - editing one paragraph in a large document doesn't require re-uploading the whole file. Each chunk is also given a hash; if two different files (or two users) contain an identical chunk, it is stored only once in **object storage** and simply referenced twice, saving significant storage space.

Sync Across Devices

Each device runs a background client that watches the local filesystem for changes. When a change is detected, the client computes which chunks changed and uploads only those, then notifies a **notification service** (built on pub/sub, Day 18) which alerts the user's other connected devices that new chunks are available to download.

Handling Conflicts

If the same file is edited on two devices while offline, both changes will eventually try to sync. Rather than silently overwriting one version, the system detects the conflicting versions (using a version number or timestamp) and keeps both, presenting the user with a "conflicted copy" file rather than guessing which change was correct.

■ Key Trade-off

Smaller chunk sizes improve deduplication and reduce data transferred on small edits, but add more bookkeeping overhead (more chunks to track per file). Larger chunks reduce that overhead but waste more bandwidth re-uploading unchanged parts of a large chunk containing even a tiny edit.

■ What Interviewers Are Really Testing

This problem tests whether you think beyond simple upload/download into the realities of syncing state across multiple, sometimes-offline devices - including how to handle conflicting edits gracefully rather than silently losing data.

PROBLEM 10

Design a Ticket Booking System (BookMyShow/Ticketmaster)

Tests concurrency control - preventing the same seat from being sold twice

■ Clarifying Questions to Ask First

Do we need seat-level selection, or just general admission tickets? Assume seat-level selection for popular events, where thousands of users might try to book the same few remaining seats simultaneously - the classic hard part of this problem.

High-Level Design

Users browse events, view a seat map, select seats, and complete payment. The seat map and availability are served from a **cache** for speed, but the actual booking - the part that must be perfectly accurate - always goes through the primary database, never the cache.

Deep Dive: Preventing Double-Booking

When a user selects a seat, the system places a short-lived **lock** (e.g., a 5-10 minute reservation hold) on that seat in the database, using an atomic "reserve if available" operation so two simultaneous requests for the same seat can never both succeed. If the user completes payment within the hold window, the seat is permanently booked; if the hold expires first (user abandoned checkout), the seat is automatically released back to available inventory.

This atomic reserve-if-available operation typically relies on a database transaction with proper row-level locking, or an atomic compare-and-set operation in a fast key-value store - the key requirement is that the "check if available" and "mark as reserved" steps happen as one indivisible operation, so no other request can slip in between them.

■ Key Trade-off

Holding a seat for too long (say, 30 minutes) frustrates other users who see "sold out" seats that are actually just sitting in someone's abandoned cart. Too short a hold (30 seconds) frustrates users who lose their selected seat mid-checkout. Most systems settle on a 5-10 minute hold as a practical balance.

Bottlenecks & Fixes

- A single extremely popular event creates a massive spike of simultaneous requests for the same small set of seats -> queue users into a virtual "waiting room" that admits a controlled number of shoppers at a time, protecting the booking database from being overwhelmed.
- Expired holds need to be released reliably -> use a background job or the database's own expiry mechanism, not something the user's browser is responsible for triggering.

■ What Interviewers Are Really Testing

This problem tests concurrency control specifically - can you identify the exact operation that must be atomic, and explain a concrete mechanism (locks, compare-and-set) rather than just saying "handle race conditions" vaguely. This is one of the most common places candidates lose points.

That completes all 10 practice problems. If you worked through each one using the 5-step framework before reading the model answer, you've now practiced the exact skill real interviews and real design reviews test: structuring an ambiguous problem, making reasonable assumptions explicit, and reasoning clearly about trade-offs rather than reciting memorized facts.